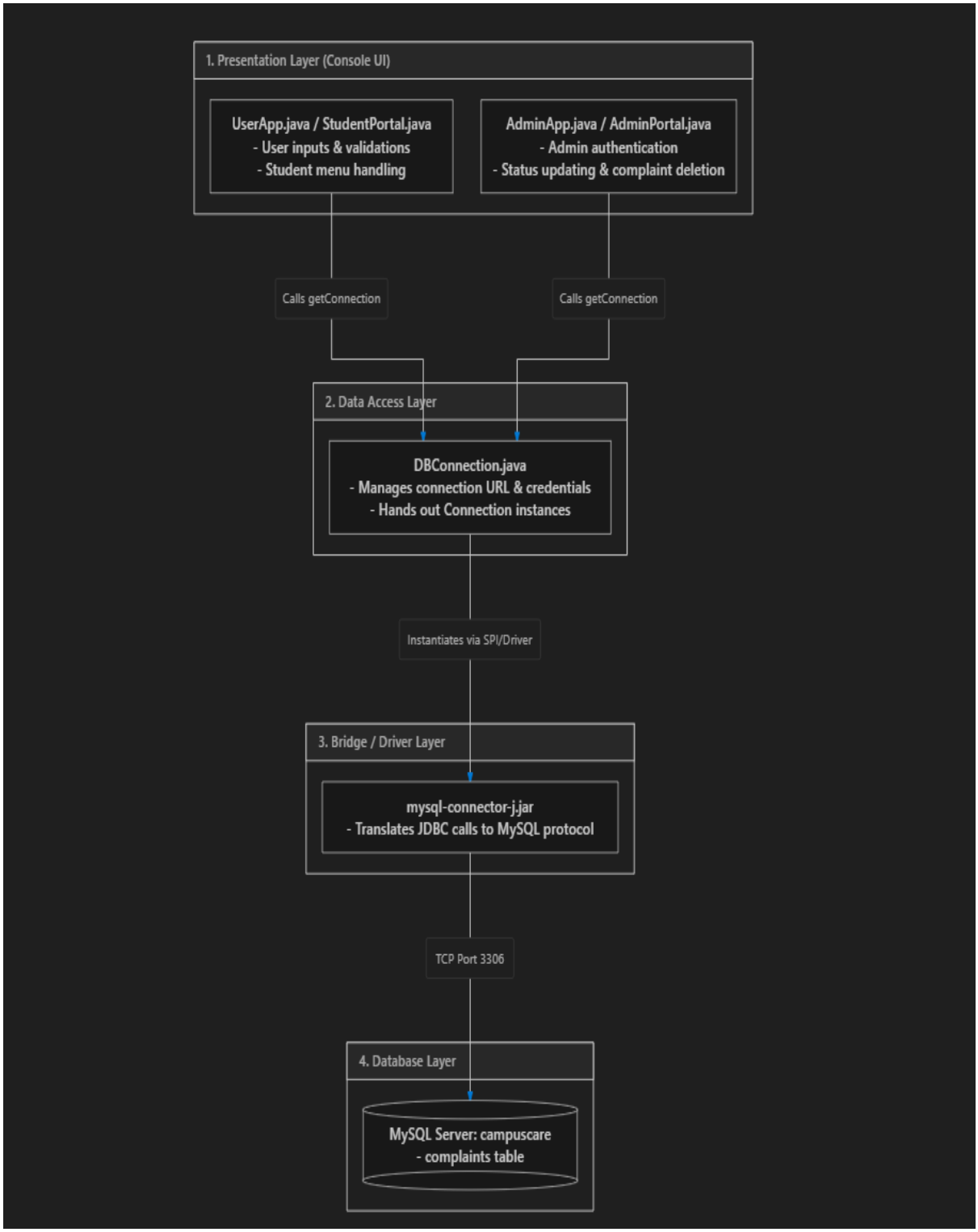
1. Overall Structure of the Project

The project implements a **Layered Architecture (3-Tier pattern)** adapted for a terminal-based console application:



Separation of Responsibilities:

1. **UserApp.java (Student Interface):**
    - Handles student interactions: reading inputs, category selection, and submitting new complaints.
    - Validates that mandatory fields (student_name, location, description) are not empty before touching the database.
2. **AdminApp.java (Admin Interface):**
    - Handles administrative tasks: viewing complaint logs, filtering by ID, advancing status (Pending →In Progress →Resolved), and deleting records.
3. **DBConnection.java (Connection Factory):**
    - Isolates configuration (database URL, port, credentials). If your MySQL password or host changes, you only update this one file instead of editing every query across the system.
4. **MySQL Database (campuscare):**
    - Stores data persistently. Relies on database-level constraints (AUTO_INCREMENT PRIMARY KEY, NOT NULL, DEFAULT CURRENT_TIMESTAMP).

---

2. Java OOP (Object-Oriented Programming) Concepts Used

Even though this is a console application, several core OOP concepts and design patterns are actively applied:

A. Encapsulation (Data Hiding & Protection)

- **Private static fields:** In DBConnection.java, the credentials (URL, USER, PASSWORD) are declared private:

java

```
private static final String URL = "jdbc:mysql://localhost:3306/campuscare";
private static final String USER = "root";
private static final String PASSWORD = "your_password";
```

Outside classes cannot view or alter database credentials directly; they can only invoke the public getConnection() method.

- **Preventing Instantiation (Utility Pattern):**

java

private DBConnection() { }

By making the constructor private, no other class can do new DBConnection(). It exists solely as a static utility.

B. Abstraction

- Abstraction means exposing *what* an object does while hiding *how* it does it.
- When UserApp.java executes:

java

Connection connection = DBConnection.getConnection();

The user interface has **zero knowledge** of TCP sockets, MySQL handshake protocols, or driver classes. It simply receives an active connection object and works with it.

- Helper methods like readRequired() and readCategory() abstract away raw Scanner input loop handling and validation.

C. Polymorphism via Interfaces

- JDBC is built heavily on **interface-driven programming**:
    - Connection, PreparedStatement, Statement, and ResultSet are all **Java Interfaces** from the java.sql package.
    - At runtime, MySQL's JAR supplies the actual concrete implementations (e.g., com.mysql.cj.jdbc.ConnectionImpl).
- Your Java code never references MySQL-specific class names. It speaks solely to the standard Java interfaces. If you switched the underlying database from MySQL to PostgreSQL or Oracle tomorrow, your application code wouldn't need to change—only the JDBC driver and connection URL.

D. Single Responsibility Principle (SRP)

- Each class has one well-defined reason to change:
    - DBConnection changes only if database network/credential settings change.
    - UserApp changes only if student UI requirements change.
    - AdminApp changes only if admin workflows change.

---

3. JDBC (Java Database Connectivity) in Detail

JDBC is Java's standardized API for database-independent connectivity. Here is how each JDBC mechanism works in this project:

1. Driver Loading & Registration

java

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

- Dynamically loads MySQL's driver class bytecode into memory.
- The static initializer inside MySQL's driver registers itself with Java's DriverManager.

2. Establishing the Connection

java

```
Connection conn = DriverManager.getConnection(URL, USER, PASSWORD);
```

- DriverManager examines the URL prefix (jdbc:mysql://).
- It hands the connection request to MySQL's registered driver.
- The driver opens a TCP network socket to localhost:3306, authenticates credentials with MySQL, and returns an active Connection object.

3. Statement vs. PreparedStatement (Security & Performance)

- **Statement:** Used in viewAllComplaints() for static queries with no user input:

java

```
Statement statement = connection.createStatement();

ResultSet result = statement.executeQuery("SELECT ... FROM complaints");
```

- **PreparedStatement:** Used whenever user input is involved (inserting, searching, updating):

java

```
String sql = "INSERT INTO complaints (student_name, category, location, description, status) VALUES (?, ?, ?, ?, 'Pending')";

PreparedStatement statement = connection.prepareStatement(sql);

statement.setString(1, studentName);

statement.setString(2, category);
```

IMPORTANT

Why PreparedStatement is critical:

1. **Prevents SQL Injection:** User input is treated strictly as literal data, never as executable SQL code.
2. **Pre-compilation:** The MySQL database server parses and compiles the SQL query structure once, caching the execution plan.

4. Executing Queries: executeQuery() vs. executeUpdate()

- **executeQuery() (DQL - Data Query Language):**
  - Used for SELECT statements.
  - Returns a ResultSet representing the database rows returned.
- **executeUpdate() (DML - Data Manipulation Language):**
  - Used for INSERT, UPDATE, and DELETE.
  - Returns an int representing the number of rows affected in the database table (e.g., 1 if an insert succeeded, 0 if no record was found to update/delete).

5. Auto-Generated Keys (Statement.RETURN_GENERATED_KEYS)

When a student registers a complaint, MySQL auto-generates the primary key id.

java

```
PreparedStatement statement = connection.prepareStatement(sql,
PreparedStatement.RETURN_GENERATED_KEYS);

statement.executeUpdate();

try (ResultSet keys = statement.getGeneratedKeys()) {
  if (keys.next()) {
    int generatedId = keys.getInt(1); // Retrieves the auto-incremented ID
  }
}
```

This saves you from having to run a separate query to look up the newly created complaint ID.

6. The ResultSet Cursor

java

```
while (result.next()) {
   int id = result.getInt("id");
   String name = result.getString("student_name");
}
```

- A ResultSet maintains an internal cursor pointing just before the first row.
- result.next() moves the cursor forward to the next row. It returns true if a row exists, or false when there are no more rows.
- Getter methods (getInt, getString, getTimestamp) extract values from columns of the current row.

7. Automatic Resource Management (try-with-resources)

Database connections, statements, and result sets consume precious operating system network sockets and MySQL server memory. If they are left open, the application suffers from **connection leaks**.

Both portals use Java 7+ **try-with-resources**:

java

```
try (Connection connection = DBConnection.getConnection();
    PreparedStatement statement = connection.prepareStatement(sql);
    ResultSet result = statement.executeQuery()) {

   // Perform database operations
} catch (SQLException e) {
   // Handle database error
}
```

Because Connection, PreparedStatement, and ResultSet implement java.lang.AutoCloseable, Java guarantees that all three will be **automatically closed in reverse order** when exiting the block, even if an exception or crash occurs.